

AMRITA VISHWA VIDYAPEETAM
AMRITA SCHOOL OF ENGINEERING, CHENNAI



19CSE204 -OBJECT ORIENTED PARADIGM

SUBMITTED BY:

SUVETHA SP

CH.EN.U4CCE22041

INPUT AND OUTPUT STATEMENTS

OBJECTIVE

To develop a simple Java application that simulates basic banking operations like accepting user details, depositing and withdrawing money, validating transactions, and displaying the final account summary using user input and output statements.

OVERVIEW:

The program accepts user input for name and initial balance, allows money to be deposited and withdrawn, checks for sufficient balance during withdrawal, and finally displays the account summary. The Scanner class is used for input, and System.out.println is used for output.

CODE :

```
import java.util.Scanner;

public class CCE22041 {

    public static void main(String[] args) {

        Scanner scanner = new Scanner(System.in);

        // Input: User's name
        System.out.print("Enter your name: ");
        String name = scanner.nextLine();

        // Input: Initial balance
        System.out.print("Enter initial balance: ");
        double balance = scanner.nextDouble();

        // Input: Amount to deposit
        System.out.print("Enter amount to deposit: ");
        double deposit = scanner.nextDouble();
        balance += deposit;

        // Input: Amount to withdraw
```

```

System.out.print("Enter amount to withdraw: ");
double withdraw = scanner.nextDouble();

// Check if withdrawal is possible
if (withdraw > balance) {
    System.out.println("Insufficient balance! Withdrawal denied.");
} else {
    balance -= withdraw;
    System.out.println("Withdrawal successful!");
}

// Output: Final account summary
System.out.println("\n--- Account Summary ---");
System.out.println("Account Holder: " + name);
System.out.println("Final Balance: ₹" + balance);

scanner.close();
}
}

```

OUTPUT:

```

C:\javanew>javac CCE22041.java

C:\javanew>java CCE22041
Enter your name: SUVETHA
Enter initial balance: 5000
Enter amount to deposit: 2000
Enter amount to withdraw: 3000
Withdrawal successful!

--- Account Summary ---
Account Holder: SUVETHA
Final Balance: ₹4000.0

```

DATA OPERATIONS

OBJECTIVE:

To create a Java application that evaluates an employee's salary and performance status using all fundamental data operations including arithmetic, relational, logical, and unary operators.

OVERVIEW

This program collects employee details such as name, basic salary, number of working days, leaves taken, and target completion status. Using these inputs, it:

- Calculates daily salary, deductions, and net salary using **arithmetic operations**.
- Applies **unary operators** to simulate reward point updates.
- Uses **relational operators** to evaluate attendance and performance conditions.
- Combines **logical operations** to determine award eligibility and HR warnings.
- Displays a detailed evaluation report.

CODE :

```
import java.util.Scanner;

public class CCE22041_DATA_OPERATIONS {

    public static void main(String[] args) {

        Scanner scanner = new Scanner(System.in);

        // Input

        System.out.print("Enter Employee Name: ");

        String name = scanner.nextLine();

        System.out.print("Enter Basic Salary: ");

        double basicSalary = scanner.nextDouble();

        System.out.print("Enter Working Days (out of 30): ");
```

```
int workingDays = scanner.nextInt();
```

```
System.out.print("Enter Leaves Taken: ");
```

```
int leaves = scanner.nextInt();
```

```
System.out.print("Has Employee Met Targets? (true/false): ");
```

```
boolean metTargets = scanner.nextBoolean();
```

```
// ----- Arithmetic Operators -----
```

```
double dailySalary = basicSalary / 30;          // Division
```

```
double salaryEarned = dailySalary * workingDays; // Multiplication
```

```
double leaveDeduction = leaves * 200;           // Subtraction
```

```
double netSalary = salaryEarned - leaveDeduction;
```

```
double remainder = (int)salaryEarned % 2;        // Modulus
```

```
// ----- Unary Operators -----
```

```
int rewardPoints = 0;
```

```
rewardPoints++; // increment
```

```
rewardPoints--; // decrement (back to 0)
```

```
// ----- Relational Operators -----
```

```
boolean fullAttendance = (leaves == 0);          // ==
```

```
boolean poorAttendance = (leaves > 5);           // >
```

```
boolean decentWork = (workingDays >= 25);        // >=
```

```
boolean lowWork = (workingDays <= 15);      // <=
boolean bonusSalary = (netSalary >= 40000); // >=
boolean bonusEligible = !(leaves != 0);     // != + !

// ----- Logical Operators -----

boolean excellentEmployee = (metTargets && fullAttendance); // &&
boolean warningRequired = (poorAttendance || lowWork);     // ||
boolean notEligibleForAward = !excellentEmployee;          // !

// ----- Output -----

System.out.println("\n--- Employee Evaluation Report ---");
System.out.println("Employee Name: " + name);
System.out.println("Daily Salary: ₹" + dailySalary);
System.out.println("Salary Earned: ₹" + salaryEarned);
System.out.println("Leave Deduction: ₹" + leaveDeduction);
System.out.println("Net Salary: ₹" + netSalary);
System.out.println("Salary is Even? (mod 2): " + (remainder == 0));

// Logical Output

System.out.println("Full Attendance: " + fullAttendance);
System.out.println("Decent Working Days: " + decentWork);
System.out.println("Poor Attendance: " + poorAttendance);
System.out.println("Met Targets: " + metTargets);
```

```

        System.out.println("\n>> Logical Evaluations:");

        System.out.println("Excellent Employee (Targets + No Leaves)? " +
excellentEmployee);

        System.out.println("Needs HR Warning (Too Many Leaves || Low Days)? " +
warningRequired);

        System.out.println("Award Eligible? " + !notEligibleForAward);

        if (bonusSalary && excellentEmployee) {

            System.out.println("  Bonus Status: Eligible");

        } else {

            System.out.println("  Bonus Status: Not Eligible");

        }

        scanner.close();

    }

}

```

OUTPUT:

```

C:\javaneu>java CCE22041_DATA_OPERATIONS
Enter Employee Name: SUVETHA
Enter Basic Salary: 30000
Enter Working Days (out of 30): 28
Enter Leaves Taken: 2
Has Employee Met Targets? (true/false): TRUE

```

```

--- Employee Evaluation Report ---
Employee Name: SUVETHA
Daily Salary: ?1000.0
Salary Earned: ?28000.0
Leave Deduction: ?400.0
Net Salary: ?27600.0
Salary is Even? (mod 2): true
Full Attendance: false
Decent Working Days: true
Poor Attendance: false
Met Targets: true

```

```

>> Logical Evaluations:
Excellent Employee (Targets + No Leaves)? false
Needs HR Warning (Too Many Leaves || Low Days)? false
Award Eligible? false
? Bonus Status: Not Eligible

```

```

C:\javaneu>java CCE22041_DATA_OPERATIONS
Enter Employee Name: SUVETHA
Enter Basic Salary: 30000
Enter Working Days (out of 30): 24
Enter Leaves Taken: 5
Has Employee Met Targets? (true/false): FALSE

```

```

--- Employee Evaluation Report ---
Employee Name: SUVETHA
Daily Salary: ?1000.0
Salary Earned: ?24000.0
Leave Deduction: ?1000.0
Net Salary: ?23000.0
Salary is Even? (mod 2): true
Full Attendance: false
Decent Working Days: false
Poor Attendance: false
Met Targets: false

```

```

>> Logical Evaluations:
Excellent Employee (Targets + No Leaves)? false
Needs HR Warning (Too Many Leaves || Low Days)? false
Award Eligible? false
? Bonus Status: Not Eligible

```

DATA TYPES

OBJECTIVE

To develop a Java-based **Hospital Patient Record System** that collects patient information, calculates the hospital bill based on the number of days admitted, applies insurance benefits, and demonstrates the use of **all primitive data types** in Java (byte, short, int, long, float, double, char, boolean).

OVERVIEW

The program collects patient information including:

- ID, contact number, gender, room charges, and admission details
- Uses byte and short for static info (ward number and year)
- Uses int, long for ID and contact info
- Uses float, double for financial computations
- Uses char for gender
- Uses boolean for insurance status

It then calculates the total bill with a 50% discount if the patient is insured.

CODE:

```
import java.util.Scanner;

public class CCE22041_DATATYPES {

    public static void main(String[] args) {

        Scanner scanner = new Scanner(System.in);

        // Integer types

        byte wardNumber = 5;           // byte

        short yearOfAdmission = 2025;  // short

        int patientID;                  // int

        long contactNumber;             // long
```



```
// Floating-point types
```

```
float roomChargePerDay;    // float
```

```
double totalBill;         // double
```

```
// Character and Boolean
```

```
char gender;              // char
```

```
boolean isInsured;        // boolean
```

```
// Input section
```

```
System.out.print("Enter Patient ID: ");
```

```
patientID = scanner.nextInt();
```

```
System.out.print("Enter Contact Number: ");
```

```
contactNumber = scanner.nextLong();
```

```
System.out.print("Enter Gender (M/F): ");
```

```
gender = scanner.next().charAt(0);
```

```
System.out.print("Enter Room Charge per Day: ");
```

```
roomChargePerDay = scanner.nextFloat();
```

```
System.out.print("Enter Number of Days Admitted: ");
```

```
int days = scanner.nextInt();
```

```
System.out.print("Is the patient insured? (true/false): ");

isInsured = scanner.nextBoolean();


// Billing logic

totalBill = roomChargePerDay * days;

if (isInsured) {

    totalBill *= 0.5; // 50% discount for insured patients

}


// Output summary

System.out.println("\n--- Patient Record Summary ---");

System.out.println("Patient ID      : " + patientID);

System.out.println("Contact Number   : " + contactNumber);

System.out.println("Gender           : " + gender);

System.out.println("Ward Number      : " + wardNumber);

System.out.println("Year of Admission : " + yearOfAdmission);

System.out.println("Room Charge/Day   : ₹" + roomChargePerDay);

System.out.println("Days Admitted     : " + days);

System.out.println("Insurance Status  : " + isInsured);

System.out.println("Total Bill        : ₹" + totalBill);


scanner.close();

}

}
```

OUTPUTS

```
C:\javanew>javac CCE22041_DATATYPES.java

C:\javanew>java CCE22041_DATATYPES
Enter Patient ID: 12345
Enter Contact Number: 9361177361
Enter Gender (M/F): F
Enter Room Charge per Day: 2500
Enter Number of Days Admitted: 4
Is the patient insured? (true/false): TRUE
```

```
--- Patient Record Summary ---
Patient ID      : 12345
Contact Number  : 9361177361
Gender          : F
Ward Number     : 5
Year of Admission : 2025
Room Charge/Day : ?2500.0
Days Admitted   : 4
Insurance Status : true
Total Bill      : ?5000.0
```

```
C:\javanew>java CCE22041_DATATYPES
Enter Patient ID: 12345
Enter Contact Number: 8637439433
Enter Gender (M/F): F
Enter Room Charge per Day: 2500
Enter Number of Days Admitted: 3
Is the patient insured? (true/false): FALSE
```

```
--- Patient Record Summary ---
Patient ID      : 12345
Contact Number  : 8637439433
Gender          : F
Ward Number     : 5
Year of Admission : 2025
Room Charge/Day : ?2500.0
Days Admitted   : 3
Insurance Status : false
Total Bill      : ?7500.0
```

CONTROL STATEMENTS

OBJECTIVE

To demonstrate the use of all control statements (if, if-else, nested if, if-else-if ladder, switch-case, for, while, do-while, break, continue) through a real-world ATM Simulation application.

OVERVIEW

ATM Simulation application

- Verified PIN using if, if-else, and nested if.
- Displayed a menu using do-while.
- Handled user choices using switch-case with if-else-if ladder.
- Processed deposits and withdrawals.
- Displayed last transactions using for loop with break and continue.
- Demonstrated while loop for retry logic (as demo).
- Exited the loop on choice = 5.

CODE

```
import java.util.Scanner;
```

```
public class CCE22041_CONTROL_STATEMENTS {
```

```
    public static void main(String[] args) {
```

```
Scanner scanner = new Scanner(System.in);

int correctPin = 1234;

int balance = 10000;


// --- if, if-else, nested if ---

System.out.print("Enter your 4-digit PIN: ");

int enteredPin = scanner.nextInt();


if (enteredPin == correctPin) {

    System.out.println("Access Granted!");

} else {

    System.out.println("Wrong PIN. Access Denied.");

    return; // Exit the program

}


// --- do-while loop: ATM menu repeats until user chooses to exit ---

int choice;

do {

    System.out.println("\n--- ATM Menu ---");

    System.out.println("1. Check Balance");

    System.out.println("2. Deposit");

    System.out.println("3. Withdraw");

    System.out.println("4. Mini Statement");

    System.out.println("5. Exit");

}
```

```
System.out.print("Enter your choice: ");

choice = scanner.nextInt();

// --- switch-case + if-else ladder inside ---

switch (choice) {

    case 1:

        System.out.println("Your balance is ₹" + balance);

        break;

    case 2:

        System.out.print("Enter deposit amount: ");

        int deposit = scanner.nextInt();

        if (deposit <= 0) {

            System.out.println("Invalid amount.");

            break;

        }

        balance += deposit;

        System.out.println("Deposited ₹" + deposit);

        break;

    case 3:

        System.out.print("Enter withdrawal amount: ");

        int withdraw = scanner.nextInt();

        if (withdraw <= 0) {
```

```
        System.out.println("Invalid amount.");
    } else if (withdraw > balance) {
        System.out.println("Insufficient funds.");
    } else {
        balance -= withdraw;
        System.out.println("Withdrawn ₹" + withdraw);
    }
    break;
```

case 4:

```
// --- for loop with break and continue ---
System.out.println("Last 5 Transactions:");
for (int i = 1; i <= 5; i++) {
    if (i == 3) continue; // skip 3rd record
    if (i == 5) break;    // stop early
    System.out.println("Transaction " + i + ": ₹" + (100 * i));
}
break;
```

case 5:

```
System.out.println("Thank you for using the ATM!");
break;
```

default:

```

        System.out.println("Invalid choice.");
    }

    // --- while loop example (just for demo) ---

    int retry = 0;

    while (retry < 0) { // won't run but added to demo

        System.out.println("Retry attempt: " + retry);

        retry++;

    }

    } while (choice != 5); // end when user chooses to exit

    scanner.close();

}

}

```

OUTPUTS

```

C:\javaneu>java CCE22041_CONTROL_STATEMENTS
Enter your 4-digit PIN: 1234
Access Granted!

--- ATM Menu ---
1. Check Balance
2. Deposit
3. Withdraw
4. Mini Statement
5. Exit
Enter your choice: 3
Enter withdrawal amount: 1500
Withdrawn ?1500

--- ATM Menu ---
1. Check Balance
2. Deposit
3. Withdraw
4. Mini Statement
5. Exit
Enter your choice: 1
Your balance is ?10000

--- ATM Menu ---
1. Check Balance
2. Deposit
3. Withdraw
4. Mini Statement
5. Exit
Enter your choice: 2
Enter deposit amount: 2000
Deposited ?2000

--- ATM Menu ---
1. Check Balance
2. Deposit
3. Withdraw
4. Mini Statement
5. Exit
Enter your choice: 4
Last 5 Transactions:
Transaction 1: ?100
Transaction 2: ?200
Transaction 4: ?400

--- ATM Menu ---
1. Check Balance
2. Deposit
3. Withdraw
4. Mini Statement
5. Exit
Enter your choice: 5
Thank you for using the ATM!

```

ACCESS MODIFIERS

OBJECTIVE

This program demonstrates the use of **all Access Modifiers** (public, private, protected, default) and **Non-Access Modifiers** (static, final, abstract) in a **University System** application.

OVERVIEW:

University System

- Models a university with departments
- Uses an abstract class for university information
- Uses a subclass for department-specific details
- Shows object creation, static counters, and inheritance

CODE :

// Abstract class with protected and final members

```
abstract class University {
```

```
    protected String universityName = "National University"; // protected
```

```
    final int establishedYear = 1950;                // final
```

```
    public void showUniversityInfo() {
```

```
        System.out.println("University: " + universityName);
```

```
        System.out.println("Established in: " + establishedYear);
```

```
    }
```

// Abstract method to be implemented by subclass

```
    abstract void showDepartmentInfo();
```

```
}
```

```
class Department extends University {
```

```
    public String departmentName;                // public
```



```
private int departmentCode;           // private

protected static int totalDepartments = 0;    // static & protected

// Constructor

public Department(String deptName, int deptCode) {

    this.departmentName = deptName;

    this.departmentCode = deptCode;

    totalDepartments++; // static variable updated

}

// Overriding abstract method

@Override

void showDepartmentInfo() {

    System.out.println("Department: " + departmentName);

    System.out.println("Department Code: " + departmentCode);

}

// Static method to show total departments

public static void showTotalDepartments() {

    System.out.println("Total Departments: " + totalDepartments);

}

}

// Main class with default (package-private) access

class UniversitySystem {

    public static void main(String[] args) {

        Department d1 = new Department("Computer Science", 101);

        Department d2 = new Department("Mechanical Engineering", 102);
```

```
d1.showUniversityInfo();  
  
d1.showDepartmentInfo();  
  
System.out.println();  
  
d2.showUniversityInfo();  
  
d2.showDepartmentInfo();  
  
// Static method call  
  
    Department.showTotalDepartments();  
  
}  
  
}
```

OUTPUT:

```
C:\javanew>javac CCE22041_ACCESS_MODIFIERS.java  
  
C:\javanew>java UniversitySystem  
University: National University  
Established in: 1950  
Department: Computer Science  
Department Code: 101  
  
University: National University  
Established in: 1950  
Department: Mechanical Engineering  
Department Code: 102  
Total Departments: 2
```

INHERITANCE

OBJECTIVE:

To demonstrate Inheritance and Polymorphism in Java through an Online Exam System that assigns marks based on question type and correctness using method overriding and runtime polymorphism.

OVERVIEW:

This program, CCE22041_OnlineExam, simulates an online test with three question types — **MCQ**, **True/False**, and **Short Answer**. A base class **Question** defines common properties, while derived classes override the **calculateMarks()** method to apply different marking schemes. It effectively shows how **OOP principles** like code reusability, method overriding, and **dynamic behavior** work in a real-world application.

CODE

```
import java.util.Scanner;
// Base class
class Question {
    protected String questionText;

    public Question(String questionText) {
        this.questionText = questionText;
    }

    // Method to calculate marks (default)
    public int calculateMarks(boolean correct) {
        return correct ? 1 : 0; // default 1 mark
    }

    public void displayResult(boolean correct) {
        System.out.println("\nQuestion: " + questionText);
        System.out.println("Correct? " + correct);
        System.out.println("Marks Awarded: " + calculateMarks(correct));
    }
}

// Derived class - Multiple Choice Question
class MCQ extends Question {
    public MCQ(String questionText) {
        super(questionText);
    }

    @Override
    public int calculateMarks(boolean correct) {
        return correct ? 2 : 0; // MCQ = 2 marks
    }
}

// Derived class - True/False Question
class TrueFalse extends Question {
```

```

public TrueFalse(String questionText) {
    super(questionText);
}

@Override
public int calculateMarks(boolean correct) {
    return correct ? 1 : 0; // True/False = 1 mark
}
}

// Derived class - Short Answer Question
class ShortAnswer extends Question {
    public ShortAnswer(String questionText) {
        super(questionText);
    }

    @Override
    public int calculateMarks(boolean correct) {
        return correct ? 5 : 0; // Short Answer = 5 marks
    }
}

// Main class with roll number included
public class CCE22041_OnlineExam {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.println("Roll Number: CCE22041");
        System.out.println("Welcome to Online Exam System!\n");

        // Input question type
        System.out.print("Enter question type (MCQ/TrueFalse/ShortAnswer): ");
        String type = scanner.nextLine();

        // Input question text
        System.out.print("Enter question text: ");
        String text = scanner.nextLine();

        // Input if student answered correctly
        System.out.print("Did the student answer correctly? (true/false): ");
        boolean correct = scanner.nextBoolean();

        Question q;

        // Polymorphism in action
        if (type.equalsIgnoreCase("MCQ")) {
            q = new MCQ(text);
        } else if (type.equalsIgnoreCase("TrueFalse")) {
            q = new TrueFalse(text);
        } else if (type.equalsIgnoreCase("ShortAnswer")) {

```

```

        q = new ShortAnswer(text);
    } else {
        q = new Question(text);
    }

    // Display result
    q.displayResult(correct);
    scanner.close();
}

```

OUTPUT :

```

C:\javaprg>javac CCE22041_ONLINE_EXAM.java
C:\javaprg>java CCE22041_ONLINE_EXAM
Roll Number: CCE22041
Welcome to Online Exam System!

Enter question type (MCQ/TrueFalse/ShortAnswer): ShortAnswer
Enter question text: Explain the concept of Encapsulation in Java.
Did the student answer correctly? (true/false): true

Question: Explain the concept of Encapsulation in Java.
Correct? true
Marks Awarded: 5

```

POLYMORPHISM

OBJECTIVE:

To demonstrate inheritance and method overriding in Java using a Vehicle Rental System that calculates rental costs for different vehicle types (Car, Bike, or Others).

OVERVIEW:

This program showcases runtime polymorphism through inheritance.

- The base class Vehicle defines the structure and a general method to calculate rental cost.
- The derived classes Car and Bike override the calculateRental() method to provide specific rates per day.
- The main class CCE22041_VehicleRentalApp allows the user to input vehicle type, name, and rental days, then displays the total rental cost accordingly.
- It also includes the roll number CCE22041 to identify the student.

CODE:

```

import java.util.Scanner;

// Base class
class Vehicle {
    protected String name;

    public Vehicle(String name) {

```

```

        this.name = name;
    }

    // Method to calculate rental cost
    public double calculateRental(int days) {
        return days * 50; // Default rate per day
    }

    public void displayRental(int days) {
        System.out.println("\n--- Rental Details ---");
        System.out.println("Vehicle: " + name);
        System.out.println("Days: " + days);
        System.out.println("Total Rental Cost: ₹" + calculateRental(days));
    }
}

// Derived class - Car
class Car extends Vehicle {
    public Car(String name) {
        super(name);
    }

    @Override
    public double calculateRental(int days) {
        return days * 100; // Car rate per day
    }
}

// Derived class - Bike
class Bike extends Vehicle {
    public Bike(String name) {
        super(name);
    }

    @Override
    public double calculateRental(int days) {
        return days * 30; // Bike rate per day
    }
}

// Main class with roll number included
public class CCE22041_VehicleRentalApp {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.println("Roll Number: CCE22041");
        System.out.println("Welcome to Vehicle Rental System!\n");

        // Input vehicle type

```

```

System.out.print("Enter vehicle type (Car/Bike/Other): ");
String type = scanner.nextLine();

// Input vehicle name
System.out.print("Enter vehicle name: ");
String name = scanner.nextLine();

// Input number of rental days
System.out.print("Enter number of days to rent: ");
int days = scanner.nextInt();

Vehicle v;

// Polymorphism in action
if (type.equalsIgnoreCase("Car")) {
    v = new Car(name);
} else if (type.equalsIgnoreCase("Bike")) {
    v = new Bike(name);
} else {
    v = new Vehicle(name);
}
v.displayRental(days);
scanner.close();
}
}

```

OUTPUT:

```

C:\javaprg>javac CCE22041_VehicleRentalApp.java
C:\javaprg>java CCE22041_VehicleRentalApp
Roll Number: CCE22041
Welcome to Vehicle Rental System!

Enter vehicle type (Car/Bike/Other): Car
Enter vehicle name: Honda City
Enter number of days to rent: 3

--- Rental Details ---
Vehicle: Honda City
Days: 3
Total Rental Cost: ₹300

```

DIFFERENT TYPES OF INHERITANCE

OBJECTIVE:

To demonstrate different types of **inheritance in Java**, including *single*, *multilevel*, *hierarchical*, and *multiple inheritance (via interfaces)* through a real-world employee management application.

OVERVIEW:

This program showcases the concept of **inheritance** by modeling relationships among different employee types:

- The **Person** class serves as the base class.
- **Employee** extends **Person** to demonstrate *single and multilevel inheritance*.
- **Manager** and **Developer** demonstrate *hierarchical inheritance* by extending **Employee**.
- **PartTimeEmployee** implements the **WorkSchedule** interface to exhibit *multiple inheritance* using an interface. The program displays details of various employee roles, showing how inheritance enables code reusability and logical organization.

CODE :

// File: CCE22041_InheritanceApp.java

// Base Class

```
class Person {
    String name;
    int age;

    Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

    void displayPersonDetails() {
        System.out.println("Name: " + name + ", Age: " + age);
    }
}
```

// Single + Multilevel Inheritance

```
class Employee extends Person {
    String empId;

    Employee(String name, int age, String empId) {
        super(name, age);
        this.empId = empId;
    }

    void displayEmployeeDetails() {
        displayPersonDetails();
        System.out.println("Employee ID: " + empId);
    }
}
```



```

}

// Hierarchical Inheritance: Manager class
class Manager extends Employee {
    String department;

    Manager(String name, int age, String empId, String department) {
        super(name, age, empId);
        this.department = department;
    }

    void displayManagerDetails() {
        displayEmployeeDetails();
        System.out.println("Department: " + department);
    }
}

// Hierarchical Inheritance: Developer class
class Developer extends Employee {
    String programmingLanguage;

    Developer(String name, int age, String empId, String programmingLanguage) {
        super(name, age, empId);
        this.programmingLanguage = programmingLanguage;
    }

    void displayDeveloperDetails() {
        displayEmployeeDetails();
        System.out.println("Programming Language: " + programmingLanguage);
    }
}

// Interface for Multiple Inheritance
interface WorkSchedule {
    void showSchedule();
}

// Multiple Inheritance using interface
class PartTimeEmployee extends Employee implements WorkSchedule {
    int workingHours;

    PartTimeEmployee(String name, int age, String empId, int workingHours) {
        super(name, age, empId);
        this.workingHours = workingHours;
    }

    @Override
    public void showSchedule() {
        System.out.println("Working Hours: " + workingHours + " per week");
    }
}

```

```

    }

    void displayPartTimeEmployeeDetails() {
        displayEmployeeDetails();
        showSchedule();
    }
}

// Main Application
public class CCE22041_InheritanceApp {
    public static void main(String[] args) {
        System.out.println("=== INHERITANCE APPLICATION DEMO ===");

        // Single & Multilevel Inheritance
        Manager m = new Manager("Alice", 40, "M101", "HR");
        System.out.println("\nManager Details:");
        m.displayManagerDetails();

        // Hierarchical Inheritance
        Developer d = new Developer("Bob", 28, "D202", "Java");
        System.out.println("\nDeveloper Details:");
        d.displayDeveloperDetails();

        // Multiple Inheritance via Interface
        PartTimeEmployee pte = new PartTimeEmployee("Charlie", 22, "P303", 20);
        System.out.println("\nPart-Time Employee Details:");
        pte.displayPartTimeEmployeeDetails();
    }
}

```

OUTPUT:

```

C:\javaprg>javac CCE22041_InheritanceApplication.java
C:\javaprg>java CCE22041_InheritanceApplication

=== INHERITANCE APPLICATION DEMO ===

Manager Details:
Name: Alice, Age: 40
Employee ID: M101
Department: HR

Developer Details:
Name: Bob, Age: 28
Employee ID: D202
Programming Language: Java

Part-Time Employee Details:
Name: Charlie, Age: 22
Employee ID: P303
Working Hours: 20 per week

```

INHERITANCE

OBJECTIVE:

The objective of this application is to demonstrate various types of inheritance in Java using a Library Management System scenario

OVERVIEW:

This Library Management System application demonstrates the concept of inheritance in Java through practical examples. It covers single, multilevel, hierarchical, and hybrid inheritance by modeling different types of library items such as books, eBooks, magazines, and borrowable books. The program highlights how subclasses can reuse and extend the functionality of base classes, and how interfaces can be combined with classes to implement hybrid inheritance. This approach improves code reusability, organization, and clarity, while providing a real-world scenario for understanding object-oriented principles.

CODE:

// Application: Library Management System (Inheritance Demonstration)

```
class LibraryItem { // Base class (Single Inheritance)
    String title;
    String author;

    LibraryItem(String title, String author) {
        this.title = title;
        this.author = author;
    }

    void displayInfo() {
        System.out.println("Title: " + title + ", Author: " + author);
    }
}

// --- Single Inheritance ---
class Book extends LibraryItem {
    String isbn;

    Book(String title, String author, String isbn) {
        super(title, author);
        this.isbn = isbn;
    }

    void showBook() {
        displayInfo();
        System.out.println("ISBN: " + isbn);
    }
}

class EBook extends Book {
    double fileSizeMB;

    EBook(String title, String author, String isbn, double fileSizeMB) {
        super(title, author, isbn);
        this.fileSizeMB = fileSizeMB;
    }
}
```

```

        void showEBook() {
            showBook();
            System.out.println("File Size: " + fileSizeMB + " MB");
        }
    }
}
class Magazine extends LibraryItem {
    int issueNumber;

    Magazine(String title, String author, int issueNumber) {
        super(title, author);
        this.issueNumber = issueNumber;
    }
    void showMagazine() {
        displayInfo();
        System.out.println("Issue Number: " + issueNumber);
    }
}
interface Borrowable {
    void borrowItem(String borrower);
}
class BorrowableBook extends Book implements Borrowable {
    boolean isBorrowed = false;

    BorrowableBook(String title, String author, String isbn) {
        super(title, author, isbn);
    }
    public void borrowItem(String borrower) {
        if (!isBorrowed) {
            isBorrowed = true;
            System.out.println(title + " has been borrowed by " + borrower);
        } else {
            System.out.println(title + " is already borrowed.");
        }
    }
}
public class CCE22041_LibraryInheritanceApp {
    public static void main(String[] args) {
        System.out.println("Roll No: CCE22041");
        System.out.println("=== LIBRARY MANAGEMENT APPLICATION (Inheritance Demo) ===\n");
        Book book = new Book("Java Basics", "James Gosling", "ISBN12345");
        System.out.println("-- Book Info (Single Inheritance) --");
        book.showBook();
        System.out.println();

        // Multilevel Inheritance
        EBook ebook = new EBook("Effective Java", "Joshua Bloch", "ISBN67890", 5.2);
        System.out.println("-- EBook Info (Multilevel Inheritance) --");
        ebook.showEBook();
        System.out.println();
    }
}

```

```

Magazine magazine = new Magazine("Tech World", "Editorial Team", 45);
System.out.println("-- Magazine Info (Hierarchical Inheritance) --");
magazine.showMagazine();
System.out.println();

// Hybrid Inheritance
BorrowableBook borrowableBook = new BorrowableBook("Clean Code", "Robert C.
Martin", "ISBN11223");
System.out.println("-- Borrowable Book (Hybrid Inheritance) --");
borrowableBook.showBook();
borrowableBook.borrowItem("Alice");
borrowableBook.borrowItem("Bob"); // Trying again
}
}

```

OUTPUT :

```

C:\javaprg>javac CCE22041_LibraryInheritanceApp.java
C:\javaprg>java CCE22041_LibraryInheritanceApp

Roll No: CCE22041
=== LIBRARY MANAGEMENT APPLICATION (Inheritance Demo) ===

-- Book Info (Single Inheritance) --
Title: Java Basics, Author: James Gosling
ISBN: ISBN12345

-- EBook Info (Multilevel Inheritance) --
Title: Effective Java, Author: Joshua Bloch
ISBN: ISBN67890
File Size: 5.2 MB

-- Magazine Info (Hierarchical Inheritance) --
Title: Tech World, Author: Editorial Team
Issue Number: 45

-- Borrowable Book (Hybrid Inheritance) --
Title: Clean Code, Author: Robert C. Martin
ISBN: ISBN11223
Clean Code has been borrowed by Alice
Clean Code is already borrowed.

```

ABSTRACTION

OBJECTIVE:

The objective of this program is to demonstrate abstraction in Java by simulating a Smart Home Appliance Control System, where abstract classes define common properties and behaviors for appliances, and concrete subclasses implement appliance-specific operations like turning on/off.

OVERVIEW

This application uses abstract classes and methods to model real-world smart home appliances. The abstract class `Appliance` contains shared attributes and abstract methods `turnOn()` and `turnOff()`. Subclasses like `Light` and `AirConditioner` implement these methods and add their unique properties. This demonstrates how abstraction hides implementation details, enforces a common interface, and allows easy extension for new appliances, improving modularity, maintainability, and code reuse.

CODE:

```
// Roll No: CCE22041
// Application: Smart Home Appliance Control System (Abstraction Demonstration)
abstract class Appliance { // Abstract Class
    String name;
    int power; // Power in Watts
    Appliance(String name, int power) {
        this.name = name;
        this.power = power;
    }

    // Abstract method to turn on the appliance
    abstract void turnOn();

    // Abstract method to turn off the appliance
    abstract void turnOff();

    // Concrete method
    void displayInfo() {
        System.out.println("Appliance: " + name + ", Power: " + power + "W");
    }
}

// Concrete class for Light
class Light extends Appliance {
    boolean isDimmer;

    Light(String name, int power, boolean isDimmer) {
        super(name, power);
        this.isDimmer = isDimmer;
    }

    @Override
    void turnOn() {
        System.out.println(name + " light is turned ON");
    }

    @Override
    void turnOff() {
        System.out.println(name + " light is turned OFF");
    }
}
```

```

void showLightDetails() {
    displayInfo();
    System.out.println("Dimmer Supported: " + isDimmer);
}
}

// Concrete class for AirConditioner
class AirConditioner extends Appliance {
    int temperature;

    AirConditioner(String name, int power, int temperature) {
        super(name, power);
        this.temperature = temperature;
    }

    @Override
    void turnOn() {
        System.out.println(name + " AC is turned ON at " + temperature + "°C");
    }

    @Override
    void turnOff() {
        System.out.println(name + " AC is turned OFF");
    }

    void showACDetails() {
        displayInfo();
        System.out.println("Temperature Set: " + temperature + "°C");
    }
}

// Main Class
public class CCE22041_AbstractionSmartHome {
    public static void main(String[] args) {
        System.out.println("Roll No: CCE22041");
        System.out.println("=== SMART HOME APPLIANCE CONTROL (Abstraction Demo)");
        System.out.println("\n");
        Light livingRoomLight = new Light("Living Room", 12, true);
        System.out.println("-- Light Info --");
        livingRoomLight.showLightDetails();
        livingRoomLight.turnOn();
        livingRoomLight.turnOff();
        System.out.println();
        AirConditioner bedroomAC = new AirConditioner("Bedroom", 1500, 24);
        System.out.println("-- Air Conditioner Info --");
        bedroomAC.showACDetails();
        bedroomAC.turnOn();
        bedroomAC.turnOff();
    }
}

```

OUTPUT :

```
C:\javaprg>javac CCE22041_AbstractionSmartHome.java
C:\javaprg>java CCE22041_AbstractionSmartHome

Roll No: CCE22041
=== SMART HOME APPLIANCE CONTROL (Abstraction Demo) ===

-- Light Info --
Appliance: Living Room, Power: 12W
Dimmer Supported: true
Living Room light is turned ON
Living Room light is turned OFF

-- Air Conditioner Info --
Appliance: Bedroom, Power: 1500W
Temperature Set: 24°C
Bedroom AC is turned ON at 24°C
Bedroom AC is turned OFF
```

ENCAPUSULATION

OBJECTIVE:

The objective of this program is to demonstrate encapsulation in Java using an Online Food Ordering System, where order details are kept private and accessed or modified only through methods, ensuring data security and controlled access.

OVERVIEW

This application models an Online Food Ordering System using encapsulation. The FoodOrder class contains private fields for customer name, food item, quantity, and price. Public methods provide controlled access to modify quantity and calculate the total price. Encapsulation ensures order details cannot be modified directly, prevents invalid input, and maintains secure and structured data handling.

CODE:

```
// Roll No: CCE22041
// Application: Online Food Ordering System (Encapsulation Demonstration)

class FoodOrder {
    // Private fields
    private String customerName;
    private String foodItem;
    private int quantity;
    private double pricePerItem;
    public FoodOrder(String customerName, String foodItem, int quantity, double pricePerItem) {
```



```

        this.customerName = customerName;
        this.foodItem = foodItem;
        this.quantity = quantity;
        this.pricePerItem = pricePerItem;
    }

    // Getter methods
    public String getCustomerName() {
        return customerName;
    }

    public String getFoodItem() {
        return foodItem;
    }

    public int getQuantity() {
        return quantity;
    }

    public double getPricePerItem() {
        return pricePerItem;
    }

    public double calculateTotal() {
        return quantity * pricePerItem;
    }

    // Setter methods
    public void setQuantity(int quantity) {
        if (quantity > 0) {
            this.quantity = quantity;
        } else {
            System.out.println("Invalid quantity!");
        }
    }

    public void displayOrder() {
        System.out.println("Customer: " + customerName + ", Food Item: " + foodItem + ",
Quantity: " + quantity + ", Price per Item: " + pricePerItem + ", Total: " + calculateTotal());
    }
}

// Main Class
public class CCE22041_EncapsulationFoodOrder {
    public static void main(String[] args) {
        System.out.println("Roll No: CCE22041");
        System.out.println("=== ONLINE FOOD ORDERING SYSTEM (Encapsulation Demo)
===\n");
    }
}

```

```
FoodOrder order1 = new FoodOrder("Alice", "Pizza", 2, 250.0);
FoodOrder order2 = new FoodOrder("Bob", "Burger", 3, 120.0);

System.out.println("-- Orders --");
order1.displayOrder();
order2.displayOrder();

System.out.println("\n-- Updating Order --");
order1.setQuantity(3); // Update quantity
order1.displayOrder();
}
}
```

OUTPUT:

```
C:\javaprg>javac CCE22041_EncapsulationFoodOrder.java
C:\javaprg>java CCE22041_EncapsulationFoodOrder

Roll No: CCE22041
=== ONLINE FOOD ORDERING SYSTEM (Encapsulation Demo) ===

-- Orders --
Customer: Alice, Food Item: Pizza, Quantity: 2, Price per Item: 250.0, Total: 500.0
Customer: Bob, Food Item: Burger, Quantity: 3, Price per Item: 120.0, Total: 360.0

-- Updating Order --
Customer: Alice, Food Item: Pizza, Quantity: 3, Price per Item: 250.0, Total: 750.0
```